

UnifiedXRMotion: A Schema-Parameterized Validity-Aware Runtime Contract for Heterogeneous XR Avatar Motion

Anonymous Authors

Paper under double-blind review

Abstract. XR avatar-motion pipelines increasingly combine directly observed tracking, vision-based landmarks, inferred body motion, replay streams, synthetic inputs, and retargeted outputs. These producers differ not only in body coverage, joint layout, and coordinate convention, but also in which pose components are available at each slot. UnifiedXRMotion explores a simple systems design decision: make component availability first-class runtime data, separate from component values. A producer publishes values into a schema-defined skeletal representation and marks which components are available; a consumer reads region views and gates access using those availability bits rather than source-specific records. Observed, inferred, replayed, synthetic, and retargeted components flow through the same representation, while provenance remains implicit in the producing node rather than encoded in the data record. Evaluation combines implementation evidence, adapter/producer coverage, reusable-consumer evidence, and a within-subjects setup-burden study. The results support a scoped systems contribution: explicit component availability can connect selected heterogeneous XR avatar-motion producers to reusable downstream consumers without requiring those consumers to depend on SDK-specific motion records.

Keywords: Extended reality · Avatar motion · Tracking middleware · Motion-data contract · Runtime systems · Authoring

1 Introduction

XR avatars depend on motion pipelines that span tracking acquisition, skeletal representation, motion processing, retargeting, recording, optional transport, and rendering. In practice these stages are often tied to one SDK or one avatar asset. A hand tracker may expose a vendor-specific joint order; a body tracker may report only a subset of joints; a vision model may provide positions but not reliable rotations; a replay stream may preserve previously recorded availability; and an inference or retargeting node may publish components that were not directly observed. This heterogeneity is not merely inconvenient engineering. It makes it hard to replace trackers, compare pipelines, record and replay motion, or reuse downstream avatar consumers without rewriting glue code.

Existing XR APIs and tracking middleware address important parts of this problem. OpenXR standardizes access to XR devices and extensions such as hand tracking [4, 5]. Earlier VR middleware, including VRPN [11], OpenTracker [9], and Ubiquitous Tracking [7], showed that device abstraction and distributed tracking are valuable architectural tools. UnifiedXRMotion builds on that lesson at a narrower layer: avatar-motion pipelines that must carry incomplete skeletal components through adapters, filters, inferred-motion producers, retargeters, record/replay utilities, and optional network wrappers.

The central design choice is to make *component availability* explicit data, separate from both schema-defined slot identity and component values. UnifiedXRMotion does not prescribe a fixed skeleton or slot count. Instead, a pipeline instantiates the contract with a schema that defines addressable motion slots, parent relations, and region views; a producer publishes values for the slots and components it has available; each pose component is guarded by a validity bit; and downstream consumers read only the components whose bits are set. This contract does not assert that a component is ground-truth correct, directly measured, or high-confidence. It only states that the producing node has made that component available for downstream consumption under its own semantics. The evaluated prototype uses one body-and-hands schema instance, but the slot count is not part of the contract semantics.

This paper makes three contributions:

- **Availability as first-class runtime data:** a schema-parameterized skeletal motion contract that separates slot identity and component values from per-component availability.
- **A heterogeneous producer and adapter boundary** that maps selected XR, vision, replay, synthetic, inferred-motion, and retargeted producers into the shared representation while localizing source-specific joint ordering, coordinate conversion, body coverage, and missing-component semantics.
- **Availability-gated downstream reuse and setup evidence**, demonstrated by retargeting, recording, and optional transport consumers that use schema views and availability bits, plus a within-subjects Unity setup-burden study for the tested authoring task.

2 Related Work

Tracking middleware and XR APIs. VRPN provides a device-independent and network-transparent interface for VR peripherals [11]. OpenTracker introduced a modular software architecture for tracking configurations [9], and Ubiquitous Tracking modeled distributed spatial relationships as a graph of tracking transformations [7]. OpenXR standardizes XR device access and extensions such as hand tracking [4, 5]; platform SDKs add body, hand, controller, camera, or device-specific streams [6]. These systems abstract devices, transforms, or API access. UnifiedXRMotion instead targets a skeletal avatar-motion record in which individual pose components may be unavailable and consumers explicitly gate reads using component availability.

Avatar motion, retargeting, and inference. Sparse avatar-motion reconstruction and motion priors have advanced rapidly [3, 2, 10]. AvatarGo studies plug-and-play self-avatar setup [8]. Textbook IK [1], codebook-style inference inspired by prior work [10], and retargeting components are used here as representative nodes that consume and republish the same motion contract. EuroXR has also highlighted reusable XR authoring abstractions, including event-driven AR design patterns [?]. UnifiedXRMotion is similar in its concern for reusable XR structure, but targets a lower-level avatar-motion boundary: component availability becomes runtime data in the motion contract rather than source-specific logic inside each consumer.

The distinction is therefore not that UnifiedXRMotion introduces device abstraction, hand-joint tracking state, retargeting, or dataflow execution independently. Prior systems provide those capabilities at different layers: tracking middleware abstracts devices and transforms, OpenXR exposes source-specific tracking states within particular API families, and avatar-rigging systems retarget motion onto a chosen character representation. UnifiedXRMotion targets a narrower boundary that becomes important when XR pipelines combine sparse device tracking, articulated hands, vision landmarks, replay streams, inferred motion, and retargeted outputs. At this boundary, heterogeneous producers need a shared avatar-motion record with schema-defined slot identity, region views, and per-component availability, while downstream consumers need to read available components without depending on the producing SDK. This paper positions availability itself as the cross-source interface, rather than treating it as a source-local tracking state or an implicit failure case inside each consumer.

Figure 1 illustrates this availability boundary using a concrete heterogeneous pipeline. In a source-specific implementation, hand tracking, vision landmarks, lower-body inference, and replay streams expose different notions of coverage, missing components, tracking state, or inferred output; that logic leaks into downstream consumers. UnifiedXRMotion moves the logic to the contract boundary: producers publish component values together with availability bits, and retargeting, recording, transport, or visualization consumers read schema-defined views using availability-gated access. The figure is conceptual and scopes the contrast to the contract boundary rather than provenance, confidence, scheduling, or latency behavior.

3 Validity-Aware Motion Contract

The central design decision in UnifiedXRMotion is to represent component availability as first-class runtime data, separate from component values. The contract instantiation that follows from this decision consists of a schema-parameterized address space, region views, per-slot pose records, and per-component availability flags. The contract does not require a fixed skeleton or slot count. Instead, a pipeline instantiates an active schema that defines slot identities, parent relations, and region views. A slot is a stable address in that schema, such as a head, wrist, fingertip, tracker, prop, facial control, or application-specific joint.

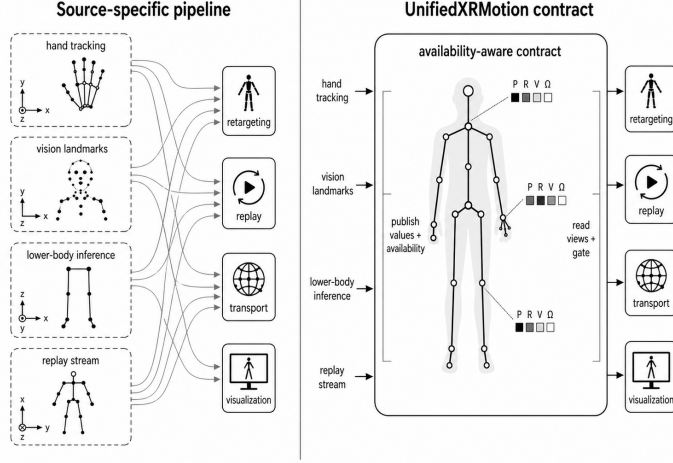


Fig. 1. Conceptual contrast for first-class component availability. Source-specific pipelines push missing-component logic into consumers, with small axis glyphs indicating source-specific coordinate and layout conventions. UnifiedXRMotion moves availability to the contract boundary: producers publish values and availability bits, and consumers read views using availability-gated access.

Runtime samples then carry component values and availability bits for the slots in the active schema. This separation is not free: it requires producer–consumer agreement on the active schema instance, and the current contract represents availability as a binary flag rather than continuous confidence.

A contract instance separates static schema from runtime state. Other applications may instantiate the same contract with different schemas, provided that producers and consumers agree on slot identities, region views, and component-availability semantics. The static schema is

$$\mathcal{S} = (S, P, R, K), \quad (1)$$

where $S = \{s_0, \dots, s_{n-1}\}$ is a finite set of motion slots, P is an optional parent relation, R is a set of named region views with each $r_j \subseteq S$, and K is the set of pose-component kinds. A runtime sample stores

$$M = \{(m_i, V_i)\}_{i=0}^{n-1}, \quad m_i = (p_i, q_i, v_i, \omega_i), \quad V_i = (b_i^p, b_i^q, b_i^v, b_i^\omega), \quad (2)$$

with $b_i^k \in \{0, 1\}$. A consumer may read component m_i^k only when $b_i^k = 1$. The prototype implements this rule through **PhysicBone**, which stores pose fields and a **PoseFlag** bitmask, and through **KinematicBuffer<T>** and **BodyType** views over full-body, upper-body, lower-body, two-arm, two-hand, and hand regions.

Table 1 summarizes how these contract concepts are instantiated in the prototype and, equally importantly, what each concept does not claim. We use the

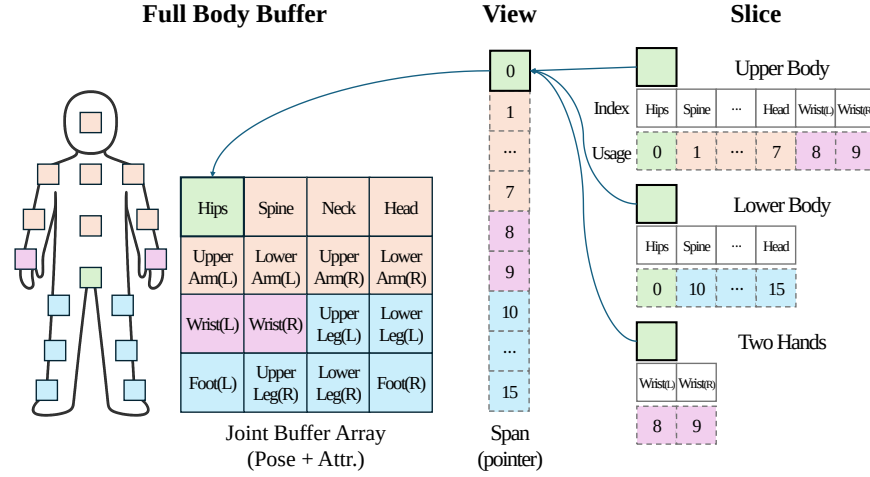


Fig. 2. Prototype buffer and region-view instantiation of the contract. An active schema buffer is exposed through region views so that producers and consumers can operate on the subset they require without fixing the contract to one slot count.

table as a traceability map between the abstract contract definition above and the concrete implementation evidence evaluated later.

Table 1. Contract concepts and evidence scope. Validity denotes component availability, not correctness, confidence, or provenance.

Concept	Prototype evidence	Scope boundary
Schema and slots	Active schema instance with slot identities, parent maps, and region views.	Slot count is not part of the contract semantics; arbitrary pipelines require schema agreement and mappings.
Pose record	Per-slot PhysicBone fields for position, rotation, velocity, scalar angular velocity, and local transforms.	The record does not carry timestamp, continuous confidence, or source provenance.
Availability	PoseFlag marks position, rotation, velocity, and angular-velocity availability per slot.	Availability does not imply ground-truth correctness or direct sensor observation.
Region views	KinematicBuffer<T> and BodyType expose full-body, upper/lower-body, arms, hands, and hand views.	Views are implementation-defined rather than discovered from arbitrary avatars.
Serialization	Optional transport writes the flag first and can omit unavailable components.	Used as semantic preservation; this paper does not frame the contribution as compression.

The contract is useful only if producers and consumers follow a small set of operational obligations. Table 2 makes these obligations explicit: producers must mark available components, consumers must gate reads, region views must preserve slot identity, and recording or transport must preserve availability together with values.

Table 2. Operational invariants used to keep the contract explicit. These are implementation-level obligations for the prototype, not claims of correctness, provenance, or uncertainty modeling.

Invariant	Required behavior	Scope boundary
Producer availability	A producer sets component flags for the values it publishes into a slot.	The bit denotes availability under producer semantics, not measured accuracy.
Consumer gating	A consumer checks the relevant flag before reading or assigning a pose component.	The paper demonstrates representative consumers rather than proving all package modules are SDK-agnostic.
View consistency	Region views preserve slot identity when exposing full-body, hand, upper-body, or lower-body subsets.	Views are derived from the active schema instance, not from a universal avatar ontology.
Availability preservation	Recording and optional transport preserve flags together with published components.	Transport is semantic support, not a latency, compression, or equivalence contribution.

These invariants are the criteria used in the following sections to discuss adapters and consumers. Adapter evidence is relevant only insofar as producer availability is set at the contract boundary, and consumer evidence is relevant only insofar as downstream nodes actually gate reads through the published flags.

This distinction is essential for inferred and retargeted motion. A lower-body inference node may publish a rotation as available, and a retargeting node may publish an avatar-specific component as available, but the contract does not encode whether that component was observed, inferred, reconstructed, replayed, or retargeted. Provenance is implicit in the producing node. Likewise, a vision adapter may threshold continuous confidence into a binary availability bit, but the continuous confidence value is not preserved in the current record. These are deliberate scope boundaries for the present prototype.

4 Heterogeneous Producers and Adapter Boundary

Producers publish motion into the contract. Device and SDK adapters are the boundary where native tracking outputs become contract records. Each adapter initializes or reads a source, maps native joint ordering into schema slots, converts coordinate conventions where needed, marks component availability, and publishes pose records. The adapter code remains vendor-specific wrapper and conversion logic; the reusable artifact is the boundary after publication, where downstream nodes depend on slot identity and component availability rather than native SDK records.

The prototype exposes `ITrackingDeviceHandler`, a minimal adapter interface for initialization, tracked schema slots, a tracking-information buffer, and a tracking update. Its producer portfolio includes OVR/Meta body and hand streams, Unity XR/OpenXR HMD, controller, and hand sources, XREAL/NRSDK and Vive hand sources, MediaPipe vision sources, Android-AR or scene-transform

sources, replay streams, synthetic sources, and representative inferred or retargeted producers. Table 3 summarizes the producer and adapter evidence used in the evaluation.

Table 3. Producer and adapter evidence. The table reports selected implementation evidence, not broad device coverage.

Producer family	Source examples	Contract mapping	Important limitation
Observed XR devices	Meta/OVR body and hand, Unity XR/OpenXR HMD/controllers/hands, Vive hands, XREAL hands.	Native joint IDs and device poses are mapped into full-body or region slots with per-component availability.	Adapter logic is SDK-specific; coverage is limited to selected source families.
Vision sources	MediaPipe upper-body and hand landmarks.	Landmark positions are mapped into hand or upper-body slots; visibility is thresholded into binary availability.	Continuous confidence is quantized away; coordinate offsets remain adapter assumptions.
Replay and synthetic sources	Recorded PhysicBone arrays, transform sources, dummy or mouse-driven inputs.	Publish the same slot records and flags as live sources.	Fidelity depends on the recorded stream or synthetic setup.
Inferred producers	Codebook-style lower/upper-body regressors and filters.	Publish inferred components back into KinematicBuffer<PhysicBone> with availability flags.	Inference algorithm and model quality are not contributions; provenance is not encoded in the record.
Retargeted producers	IK and retargeting nodes that fill avatar-motion regions.	Consume views and may republish retargeted components into the same representation.	IK and retargeting methods are representative nodes, not algorithmic novelty.

The adapter boundary localizes several classes of heterogeneity: vendor joint ordering, handedness and axis conversion, body-region coverage, missing velocity or angular-velocity channels, and SDK tracking-state or visibility conversion. Some source-specific assumptions remain visible at the adapter boundary. For example, vision visibility is reduced to a binary bit, and some device adapters contain source-specific offsets or calibration assumptions. These limitations are why the claim is limited to selected producer families and explicit component availability, not to broad cross-device coverage or automatic calibration.

5 Validity-Aware Consumers

The strongest consumer evidence is the retargeting path. **RetargetSystem** reads **KinematicBufferView<PhysicBone>** instances and checks **PoseFlag** before assigning position or rotation to avatar transforms. If a component is unavailable, the assignment is skipped rather than silently using a default or stale source value. The same consumer dispatches across **BodyType** views such as full body, upper/lower body, two arms, and two hands, so the consuming code is organized around contract views rather than around a producing SDK.

Table 4 generalizes this retargeting example to the other downstream paths included in the prototype. The table separates the contract behavior that is actually exercised from claims that remain outside the scope of the paper.

Table 4. Downstream consumers and evidence scope.

Consumer	Contract use	Supported claim	Unsupported claim
Retargeting system	Reads region views and gates transform assignment on <code>PoseFlag</code> .	Same consumer structure can handle multiple sub-skeleton views.	New retargeting algorithm or avatar-quality guarantee.
Recording/replay	Stores and republishes <code>PhysicBone</code> arrays with flags.	Replay can act as another producer in the same representation.	Ground-truth motion or accuracy benchmark.
Network serialization	Writes <code>PoseFlag</code> before optional channels and preserves availability semantics.	Optional transport can carry partial records.	Compression result, runtime-speed result, or transparent equivalence.
Filters/inference nodes	Read upstream views and republish processed or inferred components.	Observed and inferred components flow through one representation.	Encoded-provenance or uncertainty processing.

Across these consumers, the common pattern is not a shared algorithm but a shared access discipline: each consumer reads a schema-defined view and uses availability flags to decide which components may be consumed. This is the specific sense in which the prototype supports downstream reuse.

The consumer evidence focuses on representative downstream paths rather than every module in the package. Some concrete retargeters or adapters remain vendor-specific by design. The medium-paper claim is that the common contract gives downstream consumers a shared availability-gated representation, and that representative retargeting, recording, inference, replay, and transport nodes operate on that representation without depending on each native SDK record format.

Table 5 makes the reuse claim explicit by linking producer families to the contract view consumed downstream. The table is not a device-coverage benchmark; it shows that different production paths can publish into the same record/view abstraction, after which consumers branch on availability and region view rather than on the native SDK record.

6 Prototype Implementation

UnifiedXRMotion is implemented as a Unity package. Motion slots are stored in preallocated buffers; region views expose subsets without requiring a separate representation for each body part. Root producers are advanced by a frame-driven trigger list, and motion updates propagate through sender/receiver events. This runtime organization is a Unity-oriented composition mechanism rather than a general graph scheduler or graph optimizer.

This implementation realizes the contract concepts in Table 1 and the operational invariants in Table 2: buffers store slot records and flags, region views

Table 5. Producer-to-consumer reuse through shared contract views. Rows show representative paths that support the medium-paper claim; they do not imply universal source coverage or encoded provenance.

Producer path	Contract view	Downstream consumer	Claim supported
Meta/OVR body tracking	Full-body or upper/lower-body view	Retargeting system	Vendor body streams can be consumed through schema views after adapter mapping.
UnityXR/OpenXR hand sources	Two-hands or hand view	Hand retargeting and recording	Hand-only streams use the same slot/view structure as other producers.
MediaPipe vision landmarks	Position-available upper-body or hand view	Filters, recorder, optional transport	Position-dominant vision sources can publish partial component availability without requiring special downstream records.
Codebook-style inference	Lower-body or full-body view	Retargeting and recorder	Inferred components flow through the same representation, although the record does not encode inferred provenance.
Replay or synthetic source	Recorded or generated PhysicBone buffer	Same representative consumers	Replay and test inputs can act as producers without changing the consumer interface.
Retargeting or IK node	Region view republished as a buffer	Later recording or optional transport	Retargeted components can be republished as available components without claiming a new retargeting algorithm.

expose schema-defined subsets, and update paths preserve availability across producer and consumer boundaries.

At runtime, a producer updates a contract buffer for the active schema instance. Device adapters fill directly observed components, replay nodes copy recorded records, inference nodes publish estimated components, and retargeting nodes may republish transformed components into another region view. Updates are propagated through explicit sender/receiver events rather than through a global scheduler. Within each producer-triggered update path, downstream receivers consume the buffer state published by the upstream node during that event propagation. Global graph scheduling, cross-frame synchronization, out-of-order reconciliation, provenance tracking, and automatic schema negotiation are outside the scoped contract claim.

The package includes representative nodes for filtering, IK, retargeting, codebook-style inference, record/replay, and optional FishNet-based transport. Textbook IK and prior codebook-style control provide algorithmic background [1, 10]; the contribution here is that these nodes can consume or publish the same validity-aware representation. Optional transport preserves flags and available components, but transport is treated as implementation support rather than the center of the paper.

7 Evaluation and Case Study Evidence

The evaluation asks whether first-class component availability is instantiated in a working runtime, whether selected heterogeneous producers can be localized at adapters while consumers reuse contract views, and whether the resulting authoring workflow reduces procedural setup burden. The evidence is scoped to the contract boundary, producer/consumer reuse, and the prepared setup task rather than to tracking accuracy, retargeting quality, general developer productivity, broad device coverage, or display-response timing.

Contract, producers, and consumers. We use Tables 1–5 as a traceability chain for the systems claim. Table 1 identifies the contract elements and their scope boundaries; Table 2 states the operational obligations; Table 3 shows where heterogeneous sources enter the contract; Table 4 shows where availability-gated consumption is exercised; and Table 5 links representative producer paths to reusable downstream views.

Setup-burden study. We conducted a within-subjects Unity setup task comparing a UnifiedXRMotion workflow against a vendor SDK workflow. Participants configured the same scene-level goal in both conditions: making custom OpenXR hand prefabs follow simulated hand motion and making the same Y Bot avatar follow simulated full-body motion. Both conditions used the Meta XR Simulator for verification. The study used Unity 6000.0.33f1 and Meta Core, Movement SDK, and XR Simulator version 78.0.0.

The vendor-SDK condition used Meta XR Core SDK, Meta XR Interaction SDK, Movement SDK body tracking, OVRCameraRig, OVRInteractionComprehensive, synthetic hands, and the Movement SDK retargeting configuration workflow. Hand-axis alignment for the custom hand meshes was pre-completed in both conditions to avoid making axis tuning part of the measured task.

To avoid confounding pipeline assembly with mesh-specific hand-axis calibration, custom hand retargeting and axis alignment were completed before each session. Participants were instructed to prioritize correctness and completion rather than speed. We therefore interpret completion time as procedural setup burden for the tested task, not as a general developer-speed estimate. The procedure also excludes adapter authoring and full integration of a previously unseen hand mesh.

Table 6 shows lower completion time and lower workload for the UnifiedXRMotion condition, with higher SUS scores. The completion-time result had an order-sensitive pattern across counterbalanced orders (mean difference -307.6 s for AB and -521.4 s for BA; Welch test $t(16.94) = 3.32$, $p = .0040$). Sequence sensitivity was not observed for NASA-TLX ($t(16.10) = -0.31$, $p = .763$) or SUS ($t(16.99) = 1.53$, $p = .145$). The setup-burden claim is therefore useful but narrow: it supports the tested procedural configuration task, not broad claims about developer productivity.

Because the sequence effect was significant, the magnitude of the timing difference should not be interpreted as a stable estimate of general developer pro-

Table 6. Within-subjects setup-burden results ($N = 19$). Values are mean $\pm$ standard deviation. NASA-TLX is the 0–100 workload score after reversing the performance subscale so that higher values consistently indicate higher workload. Completion time is interpreted as procedural setup burden for the tested Unity task.

Statistic	Completion time (s)	NASA-TLX	SUS
UnifiedXRMotion ($M \pm SD$)	564.03 $\pm$ 176.58	30.53 $\pm$ 17.55	73.42 $\pm$ 17.08
Vendor SDK ($M \pm SD$)	972.89 $\pm$ 243.36	49.74 $\pm$ 18.29	34.47 $\pm$ 13.91
95% CI (UXM – SDK)	[–493.25, –324.48]	[–24.87, –13.55]	[28.69, 49.21]
$t(df)$	–10.18 (18)	–7.13 (18)	7.98 (18)
p	< .0001	< .0001	< .0001
Hedges’ g_{av}	–1.84	–1.03	+2.39

Statistics are computed on paired differences from the supplied raw CSVs. The study supports setup burden in this prepared task; it does not establish general developer productivity or long-term maintainability.

ductivity. The result supports the narrower claim that, in this controlled Unity setup task, the contract-based workflow reduced the procedural configuration burden required from participants.

8 Discussion and Limitations

UnifiedXRMotion is evaluated as an implemented contract boundary, not as a standardized interchange format or an end-to-end performance system. We use “runtime contract” to mean an in-memory producer–consumer representation used during pipeline execution. This is not a claim about tracker latency, display-response timing, end-to-end frame budget, or offloading performance. The core record does not encode provenance, continuous confidence, schema version, or full timestamp propagation. Provenance is implicit in the producing node, so a downstream consumer cannot tell from the data record alone whether a component was directly observed, inferred, replayed, reconstructed, or retargeted. Vision confidence is collapsed into binary availability in the current adapters, losing information that future versions should preserve.

The adapter portfolio is selected rather than exhaustive. Each new tracker still requires a schema mapping, coordinate conversion, and validation, and some mappings contain source-specific assumptions such as offsets or calibration conventions. The evaluated body-and-hands schema is one prototype instance, not a general avatar skeleton or a required slot layout. Other pipelines may instantiate different schemas, but producers and consumers must agree on slot identities, region views, and mappings to avatar-specific rigs. The included IK, retargeting, filtering, and codebook-style modules are representative nodes, not algorithmic contributions.

The setup study also has limited scope. Hand-axis alignment and mesh-specific retargeting were pre-completed, participants worked on a prepared Unity task, and the study did not measure adapter-authoring effort, long-term maintenance, or quality of produced motion. Optional local/remote traces and transport utilities characterize application-level runtime behavior only; they should

not be interpreted as display-response timing, remote-execution performance benefit, or user-perceived responsiveness.

Artifact availability. We will provide an anonymized Unity package, example scenes, analysis scripts, and the study materials used in the paper for review or upon acceptance, subject to the submission policy.

9 Conclusion

UnifiedXRMotion argues that component availability should be treated as first-class runtime data in heterogeneous XR avatar-motion pipelines. Producers publish observed, inferred, replayed, synthetic, or retargeted components into a shared schema-defined representation, and consumers read only explicitly available components through region views. The Unity prototype demonstrates this contract with selected producer families, adapter mappings, reusable retargeting/recording/transport consumers, and a controlled setup-burden study. The paper contributes a scoped contract design showing how explicit component availability can connect heterogeneous motion producers to reusable avatar-motion consumers, while leaving tracking, retargeting, inference, and scheduling algorithms to the systems that produce or consume the contract.

References

1. Aristidou, A., Lasenby, J.: FABRIK: A fast, iterative solver for the inverse kinematics problem. *Graphical Models* **73**(5), 243–260 (2011)
2. Dai, P., Zhang, Y., Liu, T., Fan, Z., Du, T., Su, Z., Zheng, X., Li, Z.: HMD-Poser: On-device real-time human motion tracking from scalable sparse observations. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. pp. 874–884 (2024)
3. Jiang, J., Strelci, P., Qiu, H., Fender, A., Laich, L., Snape, P., Holz, C.: Avatar-Poser: Articulated full-body pose tracking from sparse motion sensing. In: *Computer Vision – ECCV 2022. Lecture Notes in Computer Science*, vol. 13665, pp. 443–460. Springer (2022). https://doi.org/10.1007/978-3-031-20065-6_26, https://link.springer.com/chapter/10.1007/978-3-031-20065-6_26
4. Khronos Group: Khronos releases OpenXR 1.0 specification. <https://www.khronos.org/news/permalink/khronos-releases-openxr-1.0-specification> (2019), press release
5. Khronos Group: XR_EXT_hand_tracking. https://registry.khronos.org/OpenXR/specs/1.1/man/html/XR_EXT_hand_tracking.html (2025)
6. Meta Horizon: Openxr support for meta quest headsets. <https://developers.meta.com/horizon/documentation/native/android/mobile-intro/> (2025)
7. Newman, J., Wagner, M., Bauer, M., MacWilliams, A., Pintaric, T., Beyer, D., Pustka, D., Strasser, F., Schmalstieg, D., Klinker, G.: Ubiquitous tracking for augmented reality. In: *Proceedings of the 3rd IEEE and ACM International Symposium on Mixed and Augmented Reality*. pp. 192–201. IEEE Computer Society (2004)

8. Ponton, J.L., Monclús, E., Pelechano, N.: AvatarGo: Plug and play self-avatars for VR. In: Eurographics 2022 – Short Papers. pp. 77–80. The Eurographics Association (2022). <https://doi.org/10.2312/egs.20221037>
9. Reitmayr, G., Schmalstieg, D.: An open software architecture for virtual reality interaction. In: Proceedings of the ACM Symposium on Virtual Reality Software and Technology. ACM (2001)
10. Starke, S., Starke, P., He, N., Komura, T., Ye, Y.: Categorical codebook matching for embodied character controllers. *ACM Transactions on Graphics* **43**(4), 142:1–142:14 (2024). <https://doi.org/10.1145/3658209>
11. Taylor, R.M., Hudson, T.C., Seeger, A., Weber, H., Juliano, J., Helser, A.T.: VRPN: A device-independent, network-transparent VR peripheral system. In: Proceedings of the ACM Symposium on Virtual Reality Software and Technology. ACM (2001)